



Universidad
Indoamérica

UNIT 6 · FUNDAMENTALS OF OPERATING SYSTEMS

Signals and shared memory

Universidad Tecnológica Indoamérica Lab TD5

Signals: asynchronous notifications to a process

The kernel

SIGSEGV (invalid access), SIGBUS, SIGPIPE (pipe without a reader), SIGCHLD (a child ended), SIGALRM

The terminal

Ctrl-C → SIGINT, Ctrl-\ → SIGQUIT, Ctrl-Z → SIGTSTP, closing → SIGHUP

Another process

`kill(pid, sig)` and the `kill` command: SIGTERM, SIGKILL, SIGUSR1...

They change the process's state: they end it, stop it, resume it... or run one of its functions.

Asynchronous sending, pending signals and when they are caught

- Sending may happen at any moment, whether or not the target is running
- **Pending**: received but not yet delivered (blocked, or the kernel has not checked it yet)
- It is delivered on **returning to user mode**: after a system call or a context switch

Clarification to the original ("no signal queue exists"): it is true for *standard* signals—a pending one is a bit, repeats are lost—but **real-time** signals (`SIGRTMIN` ... `SIGRTMAX`) do queue and can carry a value (`sigqueue`).

kill and the most common signals

C

```
int kill(pid_t pid, int sig);  
/* pid > 0: that process · 0: my process group  
   < -1: group -pid · -1: every process I may signal  
   errors: EINVAL, EPERM, ESRCH */
```

Fix to the original (slide 97): it said SIGHUP, SIGINT, SIGTSTP... are "non-maskable and non-blockable". Only **SIGKILL** and **SIGSTOP** cannot be caught, ignored or blocked; all the others can.

SIGNAL	DEFAULT
SIGHUP · SIGINT · SIGTERM	terminate
SIGQUIT · SIGSEGV · SIGBUS	terminate + core
SIGKILL	terminate (always)
SIGPIPE · SIGALRM · SIGUSR1 · SIGUSR2	terminate
SIGSTOP · SIGTSTP	stop
SIGCONT	continue
SIGCHLD	ignore

Pending, blocked and delivered

signal	n	pending	blocked	action	default
SIGINT (Ctrl-C)	2	0	—	SIG_DFL	terminate
SIGQUIT (Ctrl-\)	3	0	—	SIG_DFL	terminate + core
SIGKILL	9	0	impossible	SIG_DFL	terminate
SIGUSR1	10	1	yes	handler	terminate
SIGSEGV	11	0	—	SIG_DFL	terminate + core
SIGPIPE	13	0	—	SIG_DFL	terminate
SIGALRM	14	0	—	SIG_DFL	terminate
SIGCHLD	17	0	—	SIG_DFL	ignore
SIGCONT	18	0	—	SIG_DFL	continue
SIGSTOP	19	0	impossible	SIG_DFL	stop
SIGTSTP (Ctrl-Z)	20	0	—	SIG_DFL	stop
SIGRTMIN	34	0	—	SIG_DFL	terminate

process

asleep in pause / sigsuspend

what happened

- sigprocmask(SIG_BLOCK, {SIGUSR1}); while (!flag)...
- kill(pid, SIGUSR1): the kernel marks SIGUSR1 as pending
- it arrives at the worst moment, but stays PENDING (blocked)
- sigsuspend(mask without SIGUSR1): unblocks and waits in ONE single operation

Handlers: from `signal` to `sigaction`

```
C
sighandler_t signal(int sig, sighandler_t f); /* obsolete; SIG_ERR on error */
int sigaction(int sig, const struct sigaction *act,
              struct sigaction *old);
struct sigaction {
    void (*sa_handler)(int); /* or SIG_DFL, SIG_IGN */
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask; /* blocked during the handler */
    int sa_flags; /* SA_RESTART SA_SIGINFO SA_NOCLDSTOP... */
};
```

- During the handler, the same signal stays blocked
- On return, the program resumes where it was. `sa_flags`: `SA_RESTART` restarts the interrupted call (otherwise `EINTR`, unit 2); `SA_SIGINFO` uses `sa_sigaction`; `SA_NOCLDSTOP`: `SIGCHLD` only when a child ends, not when it stops or continues
- Inside, only **async-signal-safe** functions (`write` yes, `printf` and `malloc` no) and `volatile sig_atomic_t` variables
- `signal()` has different semantics across systems: use `sigaction`

Waiting and scheduling: `pause`, `alarm`, `sleep`

- `pause()` : sleeps until a signal ends it or runs a handler; always returns -1
- `alarm(s)` : SIGALRM in `s` seconds (by default, it terminates)
- `sleep(s)` : suspends for `s` seconds

Update (slide 101): the original said `sleep` is implemented with `pause` + `alarm` and must not be mixed with them. On glibc/Linux, `sleep` uses `clock_nanosleep` and does not touch the alarm. Today `nanosleep` and `timer_create` or `timerfd` timers are preferred.

The original's examples... and their hidden bug

c

```
/* slide 102: mysleep */
signal(SIGALRM, do_nothing);
alarm(s);
pause();      /* what if SIGALRM arrives BEFORE pause? */

/* slide 103: ping-pong, parent and child after fork() */
kill(getppid(), SIGUSR1);
pause();      /* same race: the answer may come first */
```

- If the signal arrives **between** the setup and `pause()`, the handler runs early and `pause` waits forever (slide 105)
- A hard bug to detect and reproduce: it depends on the scheduler (like the `mypipe` race, unit 5)
- The original's conclusion (slide 104): **do not use** `signal()` / `pause()` to synchronize

Block, check and `sigsuspend`

C tds/td5/mysignal.c

```
/* Block SIGUSR1 outside sigsuspend: a signal arriving
   and the wait stays pending instead of being lost (th
sigemptyset(&bblock);
sigaddset(&bblock, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &bblock, &wait_mask) == -1);
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unbl
```

`sigprocmask` blocks; `sigsuspend` **unblocks and sleeps in a single atomic operation**. Set operations: `sigemptyset`, `sigfillset`, `sigaddset`, `sigdelset`, `sigismember`.

Real output

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep
0.2; kill -STOP $P; kill -USR1 $P; sleep 0.2; kill -CONT
$P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status:
$? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1
./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

Signal + memory: the sender and the receiver

Real output

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

- The receiver writes its PID at the start of a mapped file and waits for SIGUSR1 with `sigsuspend`
- The sender reads the PID, writes the message into the same mapped file and sends the signal: the signal **notifies**, the memory **carries**

Today also: `signalfd` (reading signals as data from a descriptor), `pidfd_send_signal` (signalling without reused-PID races).

Sharing memory between processes

- Virtual memory isolates: each process has its pages (unit 4)
- Sharing = making virtual pages of **two processes** point to the **same physical frame**
- A **rendezvous point** is needed: a file or a name

mmap + file

the file is the rendezvous and stores the data (persists)

shm_open +
mmap

POSIX, a name in /dev/shm (in RAM), recommended today

ftok +
shmget +
shmat

System V, numeric key; the file content is not used

Mapping a file into memory: `mmap`

C

```
void *mmap(void *addr, size_t len, int prot, int flags,  
           int fd, off_t offset); /* MAP_FAILED on error */  
int munmap(void *addr, size_t len);  
int msync(void *addr, size_t len, int flags);
```

- `prot`: `PROT_NONE`, `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`
- `flags`: `MAP_SHARED`, `MAP_PRIVATE` (copy-on-write), `MAP_ANONYMOUS`
- Errors: `EACCES` (not a regular file, or the descriptor's mode does not allow what is asked: `O_WRONLY` and `O_APPEND` do not work), `ENOMEM`

Fix to the original (slide 118): `len` does not need to be a multiple of the page size (it is rounded up); `offset` is what must be one. And accessing past the end of the file does not give `SIGSEGV` but **SIGBUS**: the next animation shows it with the real output.

Sharing memory through a file

C tds/td5/sender.c

```
int fd = open(FILENAME, O_RDWR);
exit_if(fd == -1, "open " FILENAME);
char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
exit_if(shared == MAP_FAILED, "mmap");
exit_if(close(fd) == -1, "close");

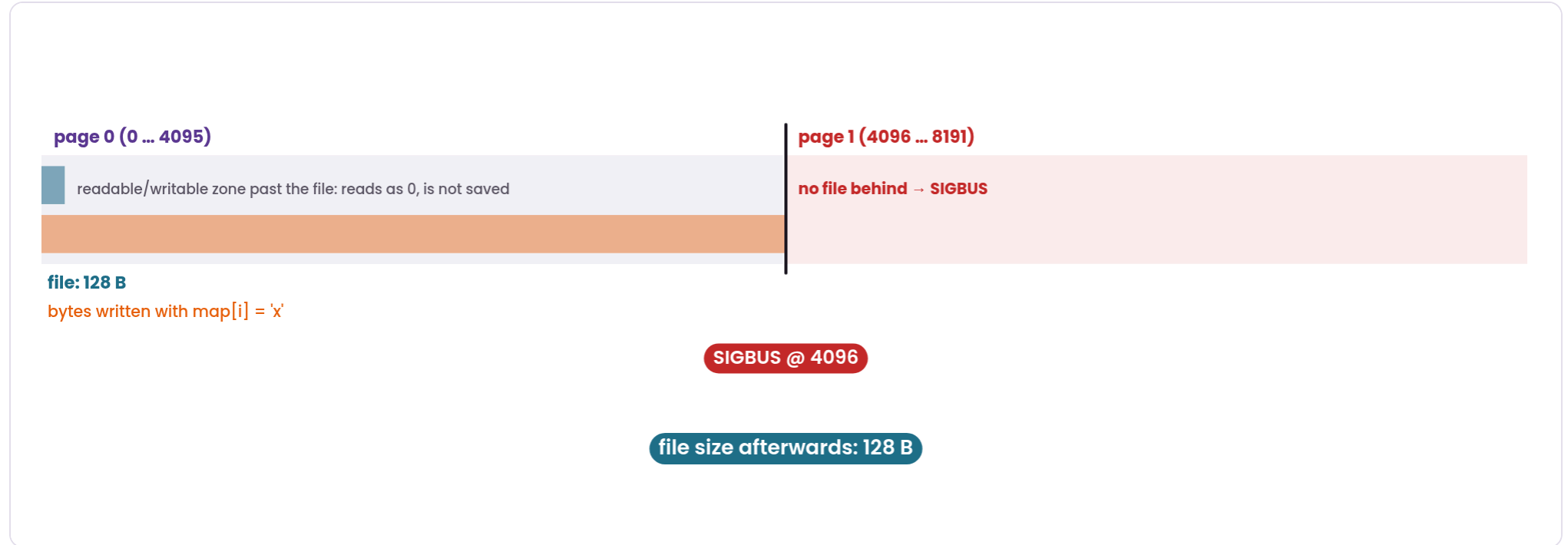
...

shared[off] = '\\0';

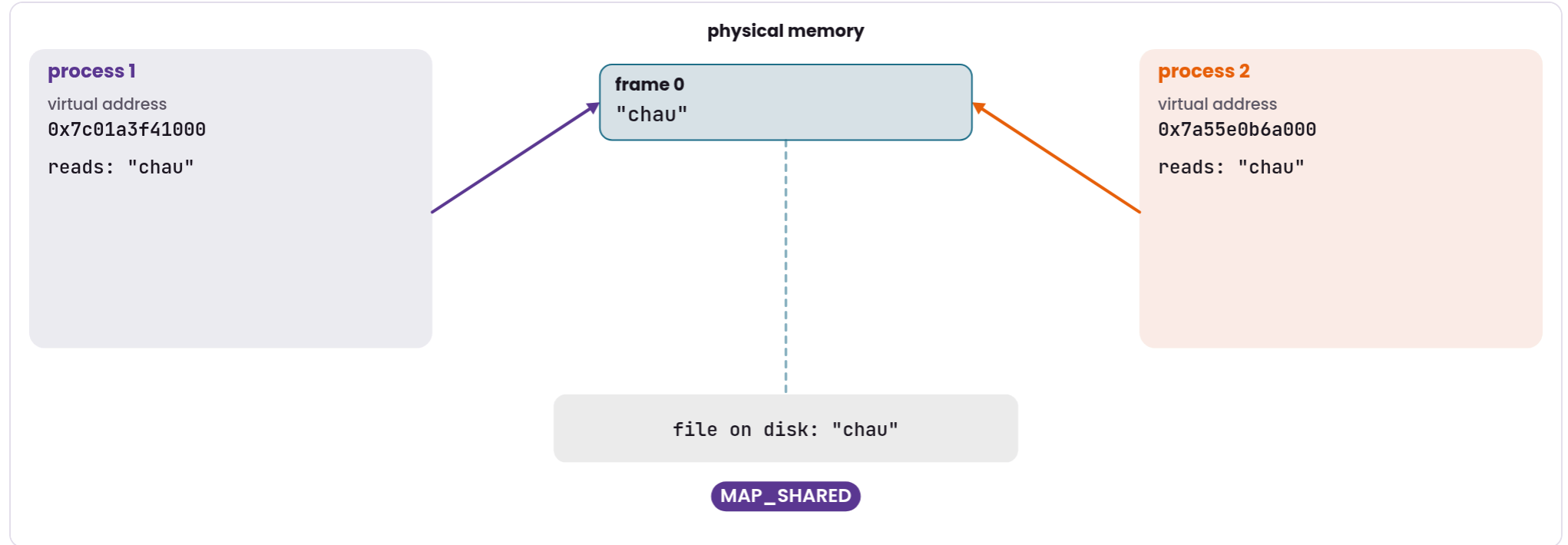
exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also push to the file */
exit_if(kill(pid, SIGUSR1) == -1, "kill");
exit_if(munmap(shared, SIZE) == -1, "munmap");
```

- `O_RDWR`: `PROT_WRITE` + `MAP_SHARED` needs a read-write descriptor
- After `mmap` the descriptor can be closed: the mapping stays alive
- Writing `shared[i]` is writing the file: the receiver, which maps the same file, sees it
- `munmap` removes the mapping; `msync` forces the copy to disk

The limits of `mmap`: `SIGBUS` and `PROT_*`



Two processes, one physical frame



System V shared memory: `ftok` → `shmget` → `shmat`

C tds/td5/shm_counter.c

```
key_t key = ftok(KEYFILE, KEYID);
exit_if(key == -1, "ftok");
int id = shmget(key, SIZE, IPC_CREAT | 0644);
exit_if(id == -1, "shmget");

if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl I");
    printf("segment %d removed\n", id);
    return EXIT_SUCCESS;
}

int *counter = shmat(id, NULL, 0);
exit_if(counter == (void *)-1, "shmat");
counter[0]++; /* a fresh
printf("segment %d (key 0x%08x): counter = %d\n", id, (
exit_if(shmdt(counter) == -1, "shmdt");
```

Real output

```
$ ./shm_counter; ./shm_counter; ./shm_counter;
./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

- The segment outlives the processes until `shmctl(IPC_RMID)`; `ipcs -m` lists them
- Each process sees it at another address: **never store pointers** in it
- `IPC_PRIVATE`: keyless segment · `SHM_RDONLY`: read-only · `shmctl`: `IPC_STAT` reads, `IPC_SET` changes permissions, `IPC_RMID` destroys

Sharing memory requires synchronizing

Without an atomic operation

```
( slot = seg[0] + 1; seg[0] = slot; ):
```

Real output

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

With `__atomic_add_fetch`:

Real output

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566
357567 357572 357568 357571 357569 357565 357574 357573
357570 357576 357578 357575 357577 357579
```

It is the topic of unit 7: race conditions, mutexes, semaphores.

Solved TD5 + AI challenge

- `mysignal` with `sigsuspend` ; the sender and the receiver
- `mmap_limits` and the five protections
- `shm_counter`, `shm_fork` and the `shm_race` race

Bash

```
make -C tds && tds/tests/run.sh td5
```



Universidad
Indoamérica

TAKE-AWAYS

Three questions

1. Why do three blocked SIGUSR1 run the handler only once?
2. What problem does `sigsuspend` solve that `pause` does not?
3. Why are pointers not stored in shared memory?